Data Structures & Systems Design Assignment

Name: Manish Kumar Singh
Programming Language: Python 3

Problem 1: LRU Cache

Approach

An LRU Cache removes the least recently used item when it reaches its capacity. I used a dictionary and a doubly linked list. The dictionary stores keys and their nodes, while the list keeps the most recently used item at the front and the least recently used item at the end. On get(), the item moves to the front. On put(), an item is added or updated, and the least recently used item is removed if the cache is full.

Python Implementation

```
class Node:
    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity):
        if capacity < 0:
            raise ValueError("Capacity cannot be negative")
        self.capacity = capacity
        self.items = {}
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_first(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.items:
            return -1
        node = self.items[key]
        self.remove(node)
        self.add_first(node)
        return node.value

    def put(self, key, value):
        if self.capacity == 0:
            return
        if key in self.items:
            node = self.items[key]
            node.value = value
            self.remove(node)
            self.add_first(node)
            return
        if len(self.items) == self.capacity:
            old_node = self.tail.prev
            self.remove(old_node)
            del self.items[old_node.key]
        node = Node(key, value)
        self.items[key] = node
        self.add_first(node)
```

Problem 2: Event Scheduler

Approach

can_attend_all() sorts events by start time and compares each event with the next one. If an event ends after the next event starts, they overlap. min_rooms_required() sorts events and uses a min-heap of ending times. Rooms whose events have ended can be reused. Events where one ends exactly as another starts do not overlap.

Python Implementation

```
import heapq


def can_attend_all(events):
    events = sorted(events)
    for i in range(1, len(events)):
        if events[i][0] < events[i - 1][1]:
            return False
    return True


def min_rooms_required(events):
    if not events:
        return 0
    events = sorted(events)
    room_ends = []
    for start, end in events:
        while room_ends and room_ends[0] <= start:
            heapq.heappop(room_ends)
        heapq.heappush(room_ends, end)
    return len(room_ends)
```

Final Discussion & Analysis

1. Time and Space Complexity

LRU get(key): Average O(1) time. Dictionary lookup and moving a node in the linked list both take constant time. Auxiliary space per operation is O(1).

LRU put(key, value): Average O(1) time. Dictionary checks and linked-list updates are constant time. The cache uses O(C) total space, where C is its capacity.

can_attend_all(events): O(n log n) time due to sorting and O(n) additional space for the sorted list.

min_rooms_required(events): O(n log n) time for sorting and heap operations, with O(n) worst-case additional space. Here, n is the number of events.

2. Why use a Hash Map and Doubly Linked List?

A hash map finds a cache item quickly using its key. A doubly linked list lets us remove a node or move it to the front without searching the list. Together, they maintain usage order while keeping get() and put() efficient.

3. How would you assign specific room numbers?

I would use two heaps. One stores occupied rooms with their ending times and room numbers; the other stores available room numbers. Before assigning a room, I would release rooms whose events have ended. If a room is available, I would reuse it; otherwise, I would create a new room. For example, if Room A becomes free at 10:00 AM and another event starts at 10:00 AM, Room A can be reused. The overall time complexity is O(n log n), with O(n) worst-case space.

4. How would you make the LRU Cache thread-safe?

I would use a lock around each public cache operation. Both get() and put() can modify the linked list, so concurrent access could otherwise leave it inconsistent. Using the same lock for the whole operation ensures that only one thread modifies the cache at a time. This is simple, though threads may need to wait for each other.

Assumptions

• LRU cache keys are hashable.
• Capacity cannot be negative; a zero-capacity cache stores nothing.
• Event intervals are valid, with start times less than or equal to end times.
• Events that end exactly when another starts do not overlap.